

Infraestructura de Identidad Segura para la FIS

Arquitectura del Sistema

La infraestructura implementa un sistema centralizado de gestion de identidades para la red de la FIS, integrando cuatro componentes sobre Ubuntu 24.04 (host WSL2 mas dos contenedores Docker): un directorio OpenLDAP que almacena usuarios, grupos y datos de directorio home bajo `dc=fis,dc=epn,dc=ec`; una autoridad certificadora ECDSA autofirmada que emite certificados de servidor para LDAP, para el servicio web protegido con Kerberos y para autenticacion entre nodos; un realm Kerberos (FIS.EPN.EC) que provee inicio de sesion unico para usuarios y servicios; y una capa de alta disponibilidad que cubre tanto el directorio como la autenticacion. El host ejecuta el servidor LDAP primario (ldap1, puerto 1388) y el KDC primario. Dos contenedores Docker, ldap2 y kdc2, ejecutan la replica LDAP secundaria y el KDC secundario respectivamente, unidos a la red del host mediante puertos mapeados (1389/1636 para ldap2, 2088/2749 para kdc2). HAProxy se ubica frente a ambos nodos LDAP como `ldap.fis.epn.edu.ec`, y Apache expone un endpoint protegido con Kerberos sobre TLS.

Diseño de la PKI

Se genero una CA raiz (CN=Servidor CA Raiz) con OpenSSL usando la curva `prime256v1`, firmada con ECDSA-SHA256, valida por diez años. Los certificados de servidor para los nodos LDAP, el servidor web y futuros servicios se emiten desde esta CA usando la misma curva, manteniendo la cadena completamente basada en curvas elípticas. OpenLDAP se configuro para escuchar en `ldaps` (636) además de `ldap`, verificado de forma independiente con `openssl s_client` contra el puerto 636 y revisando los campos `subject/issuer` con `openssl x509`. Apache termina TLS 1.3 en el puerto 443 usando un certificado emitido por la CA y asociado al FQDN `webserver.fis.epn.edu.ec`, coincidiendo con el principal de servicio Kerberos usado para SPNEGO.

Diseño de LDAP y Kerberos

El árbol del directorio separa `ou=usuarios` y `ou=grupos` bajo la raíz de la FIS; las entradas de usuario usan `inetOrgPerson` y `posixAccount`, con los atributos `uidNumber`, `gidNumber`, directorio home y shell requeridos para un backend de identidad Linux. El KDC se construyo con MIT Kerberos para el realm FIS.EPN.EC, con principals creados individualmente para cada usuario (`jperez`, `malvan`, `dnoboa`) y para los servicios (`HTTP/webserver.fis.epn.edu.ec`, `ldap/ldap1.fis.epn.edu.ec`). En lugar de migrar Kerberos a una base de datos respaldada por LDAP (el modelo de FreeIPA), la integracion se implemento importando el esquema LDAP de Kerberos (`krbPrincipalAux`, `krbPrincipalName` y tipos de atributo relacionados) en OpenLDAP y agregando un atributo `krbPrincipalName` a cada entrada de usuario, apuntando al principal Kerberos correspondiente. Esto mantiene ambas bases de datos independientes pero enlazadas de forma verificable, lo cual se valido consultando `krbPrincipalName` por LDAP para cada cuenta. Una consecuencia de este diseño, confirmada experimentalmente, es que la autenticacion Kerberos sigue funcionando aunque ambos servidores LDAP esten detenidos, ya que la emision de tickets nunca consulta el directorio en el momento de la solicitud.

El nivel web ejecuta Apache con `mod_auth_gssapi` restringido a Kerberos (`GssapiAllowedMech krb5`, sin respaldo a NTLM), protegiendo una ubicacion /seguro detras de SPNEGO. El SSO de extremo a extremo se verifico tanto desde un cliente Linux (`curl --negotiate`) como desde un cliente Windows con MIT Kerberos for Windows: tras obtener un TGT con `kinit`, Firefox accede a la pagina protegida y recibe una respuesta 200 sin ninguna solicitud de credenciales, confirmando el flujo Browser - Kerberos Ticket - Web Service requerido por el proyecto.

Arquitectura de Alta Disponibilidad

La replicacion de LDAP usa `syncrepl` en modo `refreshAndPersist`. El master ejecuta el overlay `syncprov` y una cuenta replicator dedicada con acceso de lectura al árbol; la replica obtiene los cambios con un intervalo de sondeo de 10 segundos, autenticandose con `bind simple`. Se confirmo que ambos nodos exponen un `contextCSN` identico tras una escritura en vivo en el master, y las lecturas contra la replica continuaron sin interrupcion despues de detener `slapd` en el master.

La alta disponibilidad de Kerberos usa un par de KDC primario/secundario. El secundario recibe volcados completos de la base de datos via `kprop/kpropd`, autenticados mutuamente con principals de host (`host/krb5.fis.epn.edu.ec` y `host/kdc2.fis.epn.edu.ec`) extraidos en un keytab de propagacion dedicado. El failover en esta implementacion es manual: los clientes apuntados al KDC secundario obtienen tickets validos una vez detenido el primario, pero en produccion los clientes necesitarian tener ambos KDC listados en `krb5.conf` para un failover automatico, lo cual se dejo intencionalmente como una limitacion documentada en lugar de implementarse.

El balanceo de carga coloca a HAProxy en modo TCP frente a ambos nodos LDAP, expuesto como `ldap.fis.epn.edu.ec` en el puerto 389 (el master se movio al puerto 1388 para liberar el puerto estandar para el balanceador). Los chequeos de salud corren cada dos segundos con un umbral de dos fallos antes de marcar un backend como caido. La pagina de estadisticas de HAProxy y su exportador nativo de Prometheus confirman el estado de los backends en tiempo real.

Analisis de Seguridad

Los limites de confianza siguen la jerarquia de la CA: cualquier nodo que presente un certificado no encadenado a Servidor CA Raiz es rechazado, y TLS 1.3 aborta correctamente el handshake ante un certificado expirado con una alerta `certificate_expired`, independientemente de la disponibilidad del directorio LDAP. Las credenciales Kerberos nunca viajan en texto plano por la red; las cuentas del replicator y de propagacion entre KDC usan principals dedicados y acotados a su funcion en lugar de reutilizar credenciales administrativas. El desacoplamiento entre las bases de datos de LDAP y Kerberos descrito antes es tambien una consideracion de seguridad: elimina un punto unico de falla, pero implica que los datos de directorio y autenticacion deben mantenerse sincronizados de forma deliberada, y que el compromiso de uno no compromete automaticamente al otro.

Resultados de Pruebas y Evaluacion de Rendimiento

Replicacion: una entrada nueva agregada en el master aparecio en la replica dentro de la ventana de sondeo de 10 segundos; el `contextCSN` coincidio en ambos nodos en reposo. Balanceo de carga: al detener `slapd` de forma ordenada en el master, HAProxy lo marco como caido y enruto el 100% de las lecturas hacia la replica en aproximadamente 4 segundos; al simular una caida abrupta con `kill -9` se obtuvo una ventana de recuperacion comparable, mientras que una particion de red creada con `iptables DROP` tardo cerca de 13 segundos en detectarse, ya que el chequeo de salud debe agotar su propio timeout de conexion en lugar de recibir un reset inmediato. Failover del KDC: obtener un ticket del KDC secundario tras detener el primario tomo alrededor de 3.3 segundos de principio a fin, dominado por el tiempo de tecleo de la contrasena y no por la sobrecarga del protocolo. Expiracion de certificados: se instalo un certificado firmado con fechas validas en 2020, usando la clave real del servidor, en lugar del certificado de produccion; tanto curl como Firefox rechazaron la conexion especificamente por expiracion, confirmado tanto por la alerta TLS como por el visor de certificados. Monitoreo: Prometheus recolecta metricas de `node_exporter` para CPU/memoria y del endpoint Prometheus de HAProxy para el estado de los backends y la tasa de sesiones; un script propio basado en el `textfile collector` calcula el retraso de replicacion de LDAP a partir de las marcas de tiempo del `contextCSN`, expuesto como `ldap_replication_lag_seconds` y visualizado en Grafana junto con CPU, memoria y contadores de sesiones por backend.

Dificultades de Implementacion

Dos problemas relacionados pero distintos de canonicalizacion de nombre de host consumieron la mayor parte del tiempo de depuracion del lado de Kerberos. En el host Linux, el propio `hostname` de WSL resolvia mediante `/etc/hosts` hacia un dominio asignado por el proveedor de internet en lugar de `krb5.fis.epn.edu.ec`, causando que la libreria GSSAPI construyera el nombre de principal de servicio incorrecto al propagar la base de datos de Kerberos con `kprop`; esto se soluciono configurando `rdns=false` y asignando al host su propia entrada dedicada en `hosts`. El mismo tipo de problema reaparecio despues en el cliente Windows: varios hostnames compartiendo una sola linea en el archivo `hosts` hacian que Windows devolviera el nombre canonico incorrecto para la IP compartida, por lo que el navegador solicitaba un ticket para `HTTP/fis.epn.ec` en lugar de `HTTP/webserver.fis.epn.edu.ec`; separar cada hostname en su propia linea resolvio el problema.

Una segunda fuente de fallos en la negociacion SPNEGO fue que Firefox usaba por defecto la pila SSPI nativa de Windows, la cual caia silenciosamente a NTLM una vez que MIT Kerberos for Windows emitia un ticket; dado que Apache estaba restringido intencionalmente a `GssapiAllowedMech krb5`, estos intentos eran rechazados con un 401 generico. Deshabilitar `network.auth.use-sspi` y `network.negotiate-auth.using-native-gsslib`, y apuntar `network.negotiate-auth.gsslib` directamente a la DLL de MIT Kerberos, forzo a Firefox a usar tokens Kerberos reales.

Del lado del directorio, el esquema LDAP de Kerberos tuvo que importarse manualmente tanto en el master como en la replica; cargarlo solo en el master provoco que `syncrepl` rechazara la entrada de `jperez` con un error de sintaxis en `krbPrincipalAux`, algo que solo se hizo visible en la salida de depuracion de sincronizacion de `kdc2`. Reiniciar WSL restablecia repetidamente `/etc/hosts` y en ocasiones dejaba un proceso `slapd` en estado zombie dentro de la replica Docker, que seguia reportandose como activo sin estar realmente enlazado a su puerto, lo que requirio matar el proceso manualmente antes de que la replica volviera a aceptar conexiones. Probar la expiracion de certificados tambien fallo en el primer intento porque se genero una clave nueva para el certificado de prueba en lugar de reutilizar la clave de produccion, lo que rompio la configuracion TLS de Apache; volver a generar el CSR contra la clave existente resolvio el problema.

Repositorio

El codigo fuente, los archivos de configuracion y el README con instrucciones de instalacion y pruebas estan publicados en: <https://github.com/REPLACE-WITH-USERNAME/fis-miniidm>